

Interaktiva dashboards med Streamlit

Syfte

I och med Python's kapacitet att behandla data och hämta den datan från olika källor såsom databas och kalkylark, är det önskvärt att göra något inuti Python och inte nödvändigtvis vända sig till business intelligence-program för att skapa en interaktiv dashboard på datan. Streamlit är ett bra Python bibliotek som kan öppna en webbflik med visualiserande grafer och sammanställande tabeller. Användningen av streamlit tar jag mot att inkludera drop-down menyer (filter för tabell och graf) för att lära mig om s.k. tillstånd.

Data kommer användare få lägga in i streamlits webbflik och då sköter python-skriptets logik visualer och tabeller, så användare får filtrera dem.

Området och dess relevans

Webbflikar brukar få innehåll genom programmeringsspråken HTML, JavaSkript och CSS, så att ett bibliotek kan åstadkomma samma inuti Python är relevant för datavetenskapsroller. Interaktiva dashboards underlättar framförande av trender eller datainsikter för målgrupper mindre bekanta med just programmering. Biblioteket streamlit är välkänt och låter kodbitar ta fram resultat som kan vara kunskapskrävande att återskapa i till exempel Power BI.

Att implementera logging och tester tillför i fördjupningen.

Viktiga begrepp

Körningen som görs när användarinput sker: hela skriptet körs [1]. I webbdev görs ofta lokala delar av sidan uppdaterade. Därmed kan variabler nollställas om inte specifikt behandlade.

State är informationssparande mellan skriptets körningar, ex.vis för att bevara hur ofta drop-down filter ändrats. Så länge streamlits webbflik är igång håller state anordnade variabler sparade fastän skriptet körs om när användaren interagerat. Det följer att utan state skulle en räknare av användares filterbyten max nå kvantiteten 1. [1]

Widgets som interageras med, t.ex. filuppladdning eller filterval triggar omkörning av skriptet fast med vald fil eller data inarbetat. [2]

Layout är streamlits sätt att designa dashboarden, där det går att köra grafer och tabeller inte bara vertikalt, även sida vid sida och gärna tillsammans med wide format.

Genomförande

Utifrån ett offentligt dataset *IBM HR Analytics Employee Attrition & Performance* hämtat från Kaggle kan streamlit presentera de 1,470 rader från det dataset i numeriska och grafiska representationer. Andra dataset kan också behandlas; numeriska och kategoriska kolumner sköter paketet *profiling* som skapats och anropats. Så vare sig funderingar rör sig kring en avdelnings ålderfördelning, eller hur länge kvinnor i olika åldrar haft sina chefer, kan interaktiva dashboarden med några klick ta fram detta (vilket skulle kräva flera rader kod per förfrågan utanför streamlit). Valt dataset förklaras av fokuset på streamlit; ej datastädning.

För att lämna dataset-väljande till användaren, behövs allmän behandling (motsats till hårdkodning). Pandas funktion *select_dtypes* med *include/exclude* number grupperar kolumner med allmän csv-fil kompatibilitet.

Projektets struktur syns i *README.md*. Filerna och deras syften lyder (underlättande för senare tester), och flesta finns i *profiling*-paketet:

1. *profiling.py* innehåller logiska funktioner som grupperar kolumntyper och gör sammanställningar plus validering, paketeriserat som i föreläsning 1 med mapp, *__init__.py* & *pyproject.toml*.
2. *dashboard.py* (i root) skapar användarupplevelsen (widgets och framställningar), och sköter logging vid inläsningsfel.
3. *logger_config.py* ställer in spårning i fil och terminal för spårbarhet i läsning, validering och user input. Kunskap erhöles i föreläsning 2.
4. *test_profiling.py* (i tests) sätter up pytest för dataläsning, kolumnsplitt och sammanställningar. Kunskap erhöles i föreläsning 5 och 6.

Byggandet gick till som följer. Basic *file_uploader* och visa *df*. Pandas *describe()* och *value_counts()* och övergång till streamlits plots. Filter inkorporeras genom tidig *df* enligt user input och efterföljande kod byggs på. State kan nu följa filtren och visa antalet interaktioner oavsett skriptomkörning [1]. Design av dashboard följde, dock utan brett format och kolumnisering.

Ett hinder dök upp; två filter behövs för att förändra tabell (ett för vilken kolumn och vilket värde från den kolumnen) medan fördelnings-grafen har separat dropdown för kolumn. Insikt kom då i hur streamlit hanterar widgets. Med *key* till var widget krockas de inte i state [2]. Håll graf-filtrets widget isär med *key* så plottas fördelning på rader som är kvar på separat valfri kolumn.

Resultat

Navigera en terminal till repo:t och kör *streamlit run dashboard.py* [3]. Arbetet producerar då en interaktiv webbflik med funktionerna upload .csv och automatiskt få interaktiv dashboard (välj kategori och filtrera bland värden), läsa av antal, medelvärde, *.std()* och mini- samt maximivärde, sen se kategoriernas sammansättningar och användarens antal filterinteraktioner. Först laddade jag upp en csv utan rader/kolumner, sen en csv med kolumner (ej rader), sen HR-filen.

Pytest och logging implementerades på grundlig nivå vilket säkrar resultat emot innantänkta problemscenarion samt spårar körningen. Testerna för inläsning, kolumnsplitt och sammanställningar passerade efter *pytest* körts i terminal. Just inläsning blev det aktuellt att se till att kolumner finns innan antal rader kontrolleras, för *df.empty* blir *True* båda fall. Loggingen svarade i terminal och skrev ut i loggfil.

Paketet *profiling* som skapades var viktig för att filtren ska veta vilka kolumner som är numeriska och kategoriska, vilket uppnåddes med *Pandas .select_dtypes()* och *return*, och fungerar på fler än min utvalda csv. Globala variabler bar widget keys och grafiska settings.

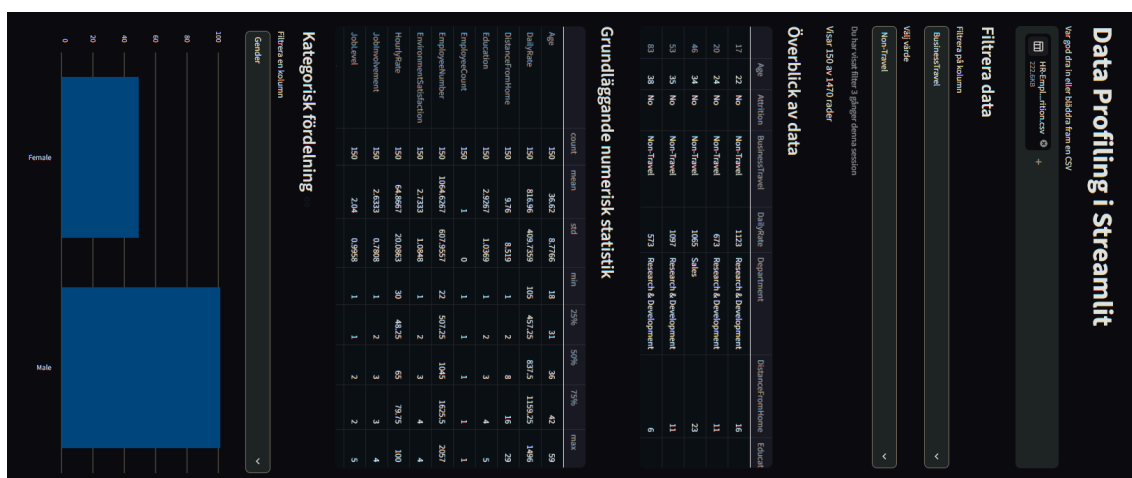


Bild 1: foto av dashboarden.

Begränsningar och möjliga förbättringar

Arbetet stöder ej databas-anslutning, vilket vore en utökning att inarbeta. Också uteslutet är machine learning och prognotiserande analys. Driften är maskinsbunden och GitHub är ett sätt att dela sitt arbete, men delning via cloud är en riktning för utökning [5].

Hantering av fel (inläsning och datavalidering) är inte total, för om upload blir csv med horisontell data kan krasch uppstå. Gränssättning i filstorlek och ytterligare logging har utvecklingspotential.

Förbättringsförslag är att ta filtren eller vertikala formatet till en egen kolumn eller brett format, och sköta synk annorlunda.

Koppling till yrkesrollen

Streamlit låter allmän kod göra specifika svar tillgängliga (händigt till svar åt kollegor som behöver många filter till data), och det är ett bra bibliotek inuti Python att göra tidiga tester eller modeller innan Power BI behövs anläggas.

Källor

- [1] <https://docs.streamlit.io/develop/concepts/architecture/session-state>
- [2] <https://docs.streamlit.io/develop/concepts/architecture/widget-behavior>
- [3] <https://docs.streamlit.io/develop/concepts/architecture/run-your-app>
- [4] IBM HR Analytics Employee Attrition & Performance, Kaggle:
<https://www.kaggle.com/pavansubhasht/ibm-hr-analytics-attrition-dataset>
- [5] <https://docs.streamlit.io/deploy/streamlit-community-cloud>

Självreflektion

1. Vad lärde du dig som du inte kunde innan?

Streamlit är annorlunda från webbdev, men har verktyg för att spara info trots att helheten körs om när användaren interagerar. Design av dashboard görs i kod, och därför behövs flertalet trial and error för att nå önskad layout.

2. Vad var svårast att förstå eller genomföra?

Motstånd möttes kring widgets och deras synk, så stegvisa tester och dokumentationsläsning påförde förståelse vilket tog arbetet vidare.

3. Vilket tekniskt val är du mest nöjd med och varför?

Select av datatyper gör inläsningen dynamisk, ökar kapaciteten för andra datafiler än HR-datan. Profilerings av datan gjordes som paket vilket bättrar modularitet och dekluttrar ``dashboard.py``, så koden enklare att förändra.

4. Vad hade du gjort annorlunda om du började om?

Det skulle gå att hålla filter i en tredje kolumn så att interaktivitet får tydligare yta.

5. Vad skulle vara ett naturligt nästa steg om du fortsatte arbetet?

Det nästa att göra är att ta analys i ytterligare riktningar, då HR-datan hade många dimensioner. Felhantering kan utökas och export-widjet för användares val av filter på datan är händig funktionalitet. Ifall inmatad csv-fil är okonventionell med värden horisontellt kan transponering vara ett led i validering.

6. Vilket betyg tycker du själv att arbetet motsvarar – G eller VG?

Kraven för godkänt anser jag levererades.

7. Motivera din bedömning genom att koppla till kraven för G och VG nedan.

Arbetet levererar kraven för godkänt, för att ämnesområdet var lagom i omfång med relevans för datavetenskap och pythonbiblioteket utforskats. Presentation och dokumentation genomfördes och kunskap till streamlit formats och förmedlats (såsom state).